

Cymbal Transit：使用 LangChain4J 和 MCP Toolbox Java SDK 的多代理系統

來源：<https://codelabs.developers.google.com/cymbal-bus-agent-mcp-toolbox-java?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將使用 LangChain4J 和 MCP Toolbox Java SDK，建構虛構的 Cymbal Bus Multiagent System。

目錄

1. [總覽](#)
2. [事前準備](#)
3. [資料庫設定](#)
4. [結構定義佈建](#)
5. [工具與工具箱設定](#)
6. [設定代理程式應用程式](#)
7. [程式碼逐步操作說明](#)
8. [MCP Toolbox：重要性和 Java SDK](#)
9. [在本機執行](#)
10. [將其部署至 Cloud Run](#)
11. [示範](#)
12. [清除所用資源](#)
13. [恭喜](#)

步驟 1 · 約 0 分鐘

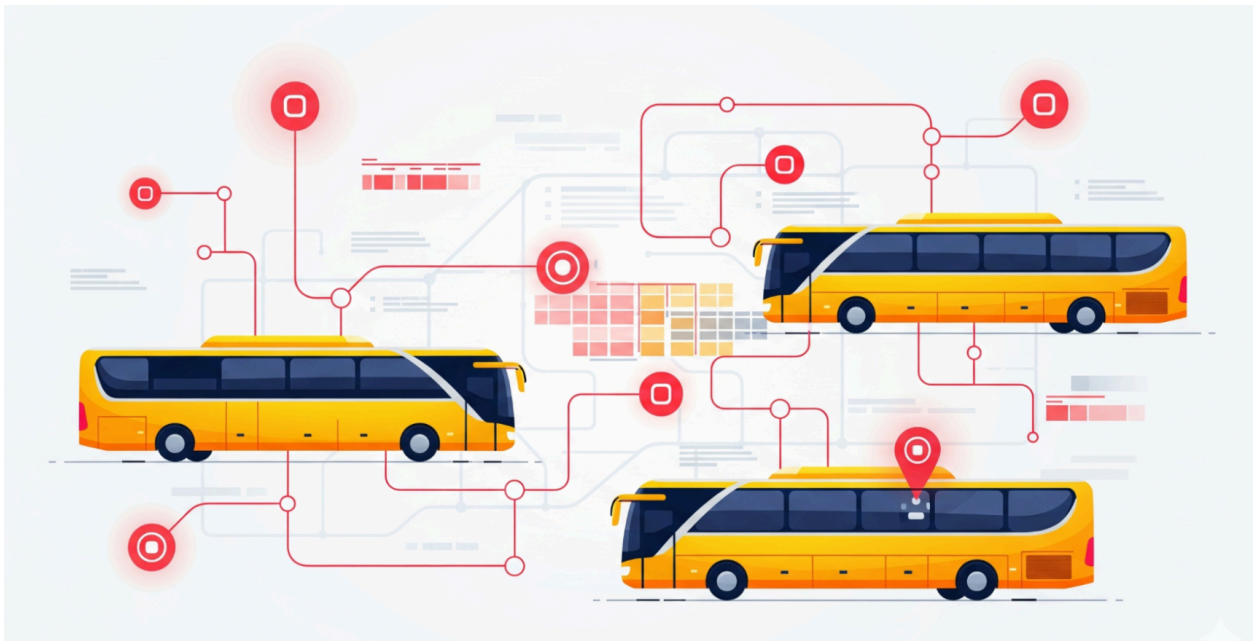
1. 總覽

現代旅客期待對話式體驗。他們不想使用複雜的 UI 篩選器，而是想問：「我可以在早上 9 點搭巴士前往波士頓，並攜帶狗狗嗎？」這需要代理程式能夠根據非結構化資料 (PDF 政策) 和結構化資料 (SQL 時間表) 進行推理。

在本實驗室中，我們將使用下列項目建構 **Cymbal Transit Agent**：

- **LangChain4j**：頂尖的 Java 框架，可自動調度管理 AI 作業。
- **AlloyDB**：與 PostgreSQL 相容的高效能資料庫。
- **MCP Toolbox Java SDK**：以標準化方式將 Java 代理連結至外部工具和資料來源。

建構項目



Cymbal Bus Agent：Java Spring Boot 應用程式，包含：

1. AlloyDB 資料庫和 MCP Toolbox Java SDK，用於協調代理程式的工具。
2. Cloud Run for Toolbox Deployment and Application (agent deployment)。
3. 在 Java 17 的 Spring Boot 應用程式中，使用 LangChain4J 程式庫做為代理程式和 LLM 架構。

課程內容

- 如何使用 LangChain4J 建立專用代理和子代理，並透過 MCP Toolbox for Databases Java SDK 協調運作
- 如何設定及使用 AlloyDB 進行資料和 AI 作業。
- 如何使用 MCP Toolbox 將代理程式連結至 AlloyDB 資料工具。
- 如何使用 Cloud Run 部署解決方案，或在本機運作執行解決方案。

架構

1. **AlloyDB for PostgreSQL**：做為高效能的作業資料庫，儲存路線、政策和預訂記錄。可支援向量搜尋和檢索。
2. **MCP Toolbox for Databases Java SDK**：做為「協調大師」，將 AlloyDB 資料公開為代理可呼叫的可執行工具。

MCP Toolbox Java SDK 可讓您輕鬆運用資料庫工具協調代理，打造企業級應用程式。

3. **LangChain4J**：開放原始碼的 Java 程式庫，可簡化大型語言模型 (LLM) 與 Java 應用程式的整合作業。這項服務提供工具和抽象化功能，可建構 AI 輔助應用程式，包括聊天機器人、代理程式和檢索增強生成 (RAG) 系統。
4. **Cloud Run**：全代管的無伺服器平台，可讓您輕鬆建構及部署應用程式或網站，並支援任何語言、程式庫和二進位檔。您可以使用慣用的語言、框架和程式庫編寫程式碼、將程式碼封裝為容器，然後執行「gcloud run deploy」指令，應用程式就能正式上線，而且系統會為應用程式提供投入正式環境所需的一切運作資源。是否建構容器全由您決定。如果您使用的是 Go、Node.js、Python、Java、.NET Core 或 Ruby，可以利用「[以來源為基礎的部署選項](#)」，透過所用語言的最佳做法來建構容器。

需求條件

- 瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
 - 已啟用計費功能的 Google Cloud 專案。
 - 對 SQL 和 Java 有基本瞭解。
-

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API：按照[這個連結](#)啟用 API。

或者，您也可以使用 gcloud 指令執行這項操作。如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

常見錯誤與疑難排解

「幽靈專案」症候群	您執行了 <code>gcloud config set project</code> ，但實際上在控制台 UI 中查看的是其他專案。檢查左上方的下拉式選單中的專案 ID！
帳單 路障	您已啟用專案，但忘記帳單帳戶。AlloyDB 是高效能引擎，如果「油箱」（帳單）空了，就無法啟動。

API 傳播 延遲	您點選了「啟用 API」，但指令列仍顯示 <code>Service Not Enabled</code> 。等待 60 秒。雲端需要一點時間喚醒神經元。
配額 Quags	如果您使用全新的試用帳戶，可能會達到 AlloyDB 執行個體的區域配額。如果 <code>us-central1</code> 失敗，請嘗試 <code>us-east1</code> 。
「隱藏」服務專員	有時系統不會自動將 <code>aiplatform.user</code> 角色授予 AlloyDB 服務代理。如果 SQL 查詢稍後無法與 Gemini 對話，通常就是這個原因。

3. 資料庫設定

我們應用程式的核心是 **AlloyDB for PostgreSQL**。我們運用了強大的向量功能和整合式資料欄引擎，為超過 **50,000 筆 SCM 記錄**生成嵌入內容。這項技術可近乎即時地進行向量分析，讓代理程式在毫秒內從大型資料集中找出庫存異常或物流風險。

在本實驗室中，我們將使用 **AlloyDB** 做為測試資料的資料庫。並使用「叢集」保存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表會保存實際資料。

接下來，我們要建立 AlloyDB 叢集、執行個體和資料表，以便載入測試資料集。

- 1. 按一下按鈕，或將下方連結複製到已登入 Google Cloud 控制台使用者的瀏覽器。

或者，您也可以前往已兌換帳單帳戶的專案，在 Cloud Shell 終端機中複製 [GitHub 存放區](#)，然後使用下列指令前往專案：

```
git clone https://github.com/AbiramiSukumaran/easy-alloydb-setup

cd easy-alloydb-setup
```

- 2. 完成這個步驟後，存放區會複製到本機 Cloud Shell 編輯器，您就能從專案資料夾執行下列指令 (請務必確認您位於專案目錄中)：

```
sh run.sh
```

- 3. 現在請使用 UI (按一下終端機中的連結，或按一下終端機中的「preview on web」連結)。
- 4. 輸入專案 ID、叢集和執行個體名稱的詳細資料，即可開始使用。
- 5. 在記錄檔捲動時去買杯咖啡吧！您可以在這裡瞭解這項功能幕後的運作方式。

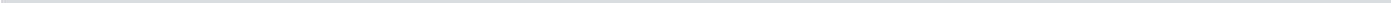
重要事項：如要在本機或任何位置進行測試，請前往 AlloyDB 執行個體，按一下「EDIT」，然後按一下「Enable Public IP」或「Public IP Connectivity」(不是輸出 IP)，並在「Authorized External Networks」中輸入「0.0.0.0/0」以供開發用途，但完成後請移除並停用公開 IP 連線。

常見錯誤與疑難排解

「耐心」問題	資料庫叢集是龐大的基礎架構，如果因為「看似卡住」而重新整理頁面或終止 Cloud Shell 工作階段，您可能會得到「虛擬」執行個體，這類執行個體已部分佈建，但必須手動介入才能刪除。
區域不符	如果您在 us-central1 中啟用 API，但嘗試在 asia-south1 中佈建叢集，可能會遇到配額問題或服務帳戶權限延遲。整個實驗室都使用同一個區域！
殭屍叢集	如果您先前使用過相同名稱的叢集，但未刪除，指令碼可能會顯示叢集名稱已存在。叢集名稱在專案內不得重複。

**Cloud Shell
超時**

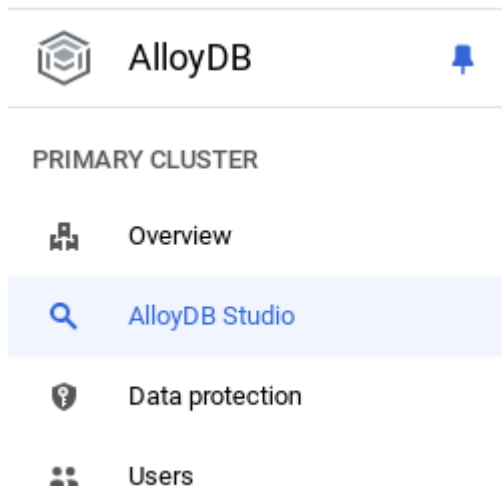
如果咖啡休息時間為 30 分鐘，Cloud Shell 可能會進入休眠狀態，並中斷 `sh run.sh` 程序。請保持分頁啟用狀態！



步驟 4 · 約 0 分鐘

4. 結構定義佈建

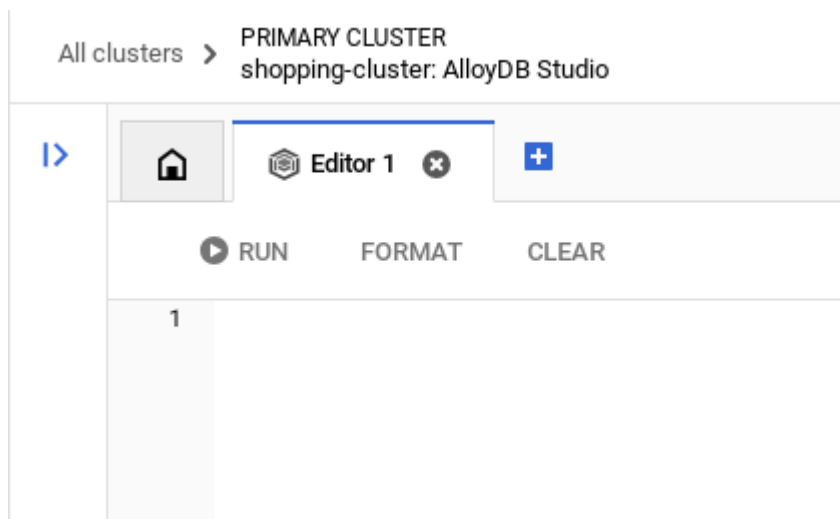
AlloyDB 叢集和執行個體啟動後，請前往 AlloyDB Studio SQL 編輯器啟用 AI 擴充功能，並佈建結構定義。



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」(或您在建立時設定的密碼)

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

啟用擴充功能

我們會使用 `pgvector` 和 `google_ml_integration` 擴充功能建構這個應用程式。[pgvector 擴充功能](#)可讓您儲存及搜尋向量嵌入。[google_ml_integration](#) 擴充功能提供多種函式，可存取 Vertex AI 預測端點，在 SQL 中取得預測結果。執行下列 DDL，[啟用](#)這些擴充功能：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;
```

授予權限

執行下列陳述式，授予「embedding」函式的執行權：

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

為 AlloyDB 服務帳戶授予 Vertex AI 使用者角色

在 **Google Cloud IAM** 控制台中，將「Vertex AI 使用者」角色授予 AlloyDB 服務帳戶 (看起來像這樣：service-
<<PROJECT_NUMBER>>@gcp-sa-alloydb.iam.gserviceaccount.com)。PROJECT_NUMBER 會顯示您的專案編號。

或者，您也可以從 Cloud Shell 終端機執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)  
  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --  
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

建立資料表

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
DROP TABLE IF EXISTS transit_policies;
DROP TABLE IF EXISTS bus_schedules;
DROP TABLE IF EXISTS bookings;

-- Table 1: Transit Policies (Unstructured Data for RAG)
CREATE TABLE transit_policies (
    policy_id SERIAL PRIMARY KEY,
    category VARCHAR(50),
    policy_text TEXT,
    policy_embedding vector(768)
);

-- Table 2: Intercity Bus Schedules (Structured Data)
CREATE TABLE bus_schedules (
    trip_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    origin_city VARCHAR(100),
    destination_city VARCHAR(100),
    departure_time TIMESTAMP,
    arrival_time TIMESTAMP,
    available_seats INT DEFAULT 50,
    ticket_price DECIMAL(6,2)
);

-- Table 3: Booking Ledger (Transactional Action Data)
CREATE TABLE bookings (
    booking_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trip_id UUID REFERENCES bus_schedules(trip_id),
    passenger_id VARCHAR(100),
    status VARCHAR(20) DEFAULT 'CONFIRMED',
    booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`policy_embedding` 欄可儲存部分文字欄位的向量值。

資料擷取

執行下列 SQL 陳述式集，在各自的資料表中大量插入記錄：

1. 在 AlloyDB 中插入非結構化政策，並以原生方式生成真實嵌入

```
-- 1. Insert Unstructured Policies and GENERATE REAL EMBEDDINGS natively in AlloyDB

INSERT INTO transit_policies (category, policy_text, policy_embedding)
VALUES
('Pets', 'Service animals are always welcome. Small pets (under 25 lbs) are allowed in secure carriers for a $25 fee. Large dogs are not permitted on standard coaches.', embedding('text-embedding-005', 'Service animals are always welcome. Small pets (under 25 lbs) are allowed in secure carriers for a $25 fee. Large dogs are not permitted on standard coaches.')),
('Luggage', 'Each passenger is allowed one carry-on (up to 15 lbs) and two stowed bags (up to 50 lbs each) free of charge. Additional bags cost $15 each.', embedding('text-embedding-005', 'Each passenger is allowed one carry-on (up to 15 lbs) and two stowed bags (up to 50 lbs each) free of charge. Additional bags cost $15 each.')),
('Refunds', 'Tickets are fully refundable up to 24 hours before departure. Within 24 hours, tickets can be exchanged for travel credit only.', embedding('text-embedding-005', 'Tickets are fully refundable up to 24 hours before departure. Within 24 hours, tickets can be exchanged for travel credit only.'));
```

2. 使用 generate_series 為 7 天生成 200 多個實際時間表

```
-- 2. Generate 200+ Realistic Schedules for the Next 7 Days using generate_series

INSERT INTO bus_schedules (origin_city, destination_city, departure_time, arrival_time,
ticket_price, available_seats)
SELECT
    origin,
    destination,
    -- Generate departures every 4 hours starting from tomorrow
    (CURRENT_DATE + 1) + (interval '4 hours' * seq) AS dep_time,
    (CURRENT_DATE + 1) + (interval '4 hours' * seq) + interval '4.5 hours' AS arr_time,
    ROUND((RANDOM() * 30 + 25)::numeric, 2) AS price, -- Random price between $25 and $55
    FLOOR(RANDOM() * 50 + 1) AS seats -- Random seats between 1 and 50
FROM
    (VALUES
        ('New York', 'Boston'), ('Boston', 'New York'),
        ('Philadelphia', 'Washington DC'), ('Washington DC', 'Philadelphia'),
        ('Seattle', 'Portland'), ('Portland', 'Seattle')
    ) AS routes(origin, destination)
CROSS JOIN generate_series(1, 40) AS seq; -- 6 routes * 40 time slots = 240 distinct trips
ingested!
```

生成嵌入

使用「`embedding('text-embedding-005', '<<policytext>>')`」函式將插入陳述式自動嵌入 `transit_policies` 表格。

常見錯誤與疑難排解

「密碼失憶」迴圈	如果您使用「一鍵」設定，但忘記密碼，請前往控制台的「執行個體基本資訊」頁面，然後按一下「編輯」重設 <code>postgres</code> 密碼。
----------	---

「找不到擴充功能」錯誤	如果 <code>CREATE EXTENSION</code> 失敗，通常是因為執行個體仍處於初始佈建的「維護」或「更新」狀態。檢查執行個體建立步驟是否完成，視需要等待幾秒鐘。
IAM 傳播問題	您已執行 <code>gcloud</code> IAM 指令，但 <code>SQL CALL</code> 仍因權限錯誤而失敗。IAM 變更可能需要一段時間才會透過 Google 骨幹網路傳播。深呼吸。***重要事項： - 有時，AlloyDB 服務帳戶可能與權限步驟中使用的現有格式不同。為確保 AlloyDB 服務帳戶具備 Vertex AI 使用者角色：前往 Google Cloud 控制台的「AlloyDB Clusters」(AlloyDB 叢集) 頁面。按一下叢集，然後在「總覽」分頁中，尋找標示為「服務帳戶」的欄位。 複製該值，然後前往 IAM 並新增 Vertex AI 使用者角色。 - 此外，如果您略過「開始前」一節中的「啟用 API」步驟，存取 AlloyDB 中的嵌入內容時會遇到問題。
向量維度不符	<code>transit_policies</code> 資料表的 <code>policy_embedding</code> 資料欄設為 <code>VECTOR(768)</code> 。如果之後嘗試使用其他模型 (例如 1536 維度模型)，插入內容就會爆炸。請使用 <code>text-embedding-005</code> 。
專案 ID 拼字錯誤	在 <code>create_model</code> 呼叫中，如果保留方括號 <code>« »</code> 或專案 ID 輸入錯誤，模型註冊看起來會成功，但第一次實際查詢時會失敗。請仔細檢查字串！

步驟 5 · 約 0 分鐘

5. 工具與工具箱設定

MCP Toolbox for Databases 是資料庫專用的開放原始碼 MCP 伺服器。這項服務可處理連線集區、驗證和[其他](#)複雜作業，讓您更輕鬆、快速且安全地開發工具。您可以透過 Toolbox 建構生成式 AI 工具，讓代理程式存取資料庫中的資料。

我們使用 **Model Context Protocol (MCP) Toolbox for Databases** 做為「指揮家」。這項服務可做為代理程式與 AlloyDB 之間的標準化中介軟體。定義 `tools.yaml` 設定後，工具箱會自動將複雜的資料庫作業公開為可執行的乾淨工具，例如 `find-bus-schedules and routes` 或 `query-schedules for specific routes`，並執行 `book-ticket` 等自主動作。這樣就不必在代理程式邏輯中手動建立連線集區或樣板 SQL。

安裝 Toolbox 伺服器

在 Cloud Shell 終端機中，建立資料夾來儲存新的工具 YAML 檔案和工具箱二進位檔：

```
mkdir cymbal-bus-toolbox

cd cymbal-bus-toolbox
```

在該新資料夾中，執行下列指令集：

```
# see releases page for other versions
export VERSION=0.27.0
curl -L -o toolbox https://storage.googleapis.com/mcp-toolbox-for-
databases/v$VERSION/linux/amd64/toolbox
chmod +x toolbox
```

接著，在該新資料夾中建立 **tools.yaml** 檔案，方法是前往 Cloud Shell 編輯器，然後將這個 [repo](#) 檔案的內容複製到 `tools.yaml` 檔案。

... (Refer to entire file in the repo)

tools:

find-bus-schedules:

kind: postgres-sql

source: alloydb

description: Find all available bus schedules.

statement: |

```
SELECT CAST(trip_id AS TEXT) trip_id, departure_time, arrival_time, ticket_price,
available_seats , origin_city, destination_city
FROM bus_schedules;
```

query-schedules:

kind: postgres-sql

source: alloydb

description: Find available bus schedules between an origin and destination city.

parameters:

- name: origin
type: string
description: The departure city name.
- name: destination
type: string
description: The arrival city name.

statement: |

```
SELECT CAST(trip_id AS TEXT) trip_id, departure_time, arrival_time, ticket_price,
available_seats
FROM bus_schedules
WHERE lower(origin_city) = lower($1)
AND lower(destination_city) = lower($2)
AND available_seats > 0
ORDER BY departure_time ASC
LIMIT 5;
```

book-ticket:

kind: postgres-sql

source: alloydb

description: Books a ticket for a specific trip, decrementing available seats and generating a confirmed booking record.

parameters:

- name: trip_id
type: string
description: The UUID of the trip schedule to book.
- name: passenger_name
type: string
description: Name or ID of the passenger (Bound securely via backend or AuthToken).
authServices:
 - name: google_auth
field: sub

statement: |

```
WITH updated_schedule AS (
UPDATE bus_schedules
```

```

        SET available_seats = available_seats - 1
        WHERE trip_id = CAST($1 AS UUID) AND available_seats > 0
        RETURNING trip_id
    )
    INSERT INTO bookings (trip_id, passenger_id)
    SELECT trip_id, $2
    FROM updated_schedule
    RETURNING CAST(booking_id as TEXT) as booking_id, trip_id, passenger_id, status,
booking_time;

search-policies:
  kind: postgres-sql
  source: alloydb
  description: Semantic search for transit policies regarding luggage, pets, refunds, and
general rules.
  parameters:
    - name: search_query
      type: string
      description: The user's question about transit policies to be embedded and searched.
  statement: |
    SELECT category, policy_text
    FROM transit_policies
    ORDER BY policy_embedding <=> CAST(embedding('text-embedding-005', $1) AS vector(768))
    LIMIT 2;

```

注意：

1. 在 tools.yaml 設定中，請務必在 alloydb 來源設定中加入 **ipType: "private"**。
2. 此外，請記得在 authServices 設定的 clientId 參數中加入 MCP Toolbox 服務網址。您可能要等到初始部署完成後才會收到連結，因此您必須執行兩次部署步驟，確保已驗證的工具使用案例正常運作。
3. 如果 AlloyDB 連線設為私有，以下本機測試工具箱的選項**將無法運作**，您必須將連線設為公開，才能在本機測試，或使用 Proxy 連線。但請**不必擔心**，在本例中，我們會直接部署至 Cloud Run，然後進行測試。

如要在本機伺服器中測試 tools.yaml 檔案，請執行下列步驟：

```
./toolbox --tools-file "tools.yaml"
```

您也可以 UI 中測試：

```
./toolbox --ui
```

請按照下列步驟，在 Cloud Run 中部署。

Cloud Run 部署作業

1. 設定 PROJECT_ID 環境變數：

```
export PROJECT_ID="my-project-id"
```

2. 初始化 gcloud CLI：


```
gcloud init
gcloud config set project $PROJECT_ID
```

3. 您必須啟用下列 API：

```
gcloud services enable run.googleapis.com \
    cloudbuild.googleapis.com \
    artifactregistry.googleapis.com \
    iam.googleapis.com \
    secretmanager.googleapis.com
```

4. 如果沒有後端服務帳戶，請建立一個：

```
gcloud iam service-accounts create toolbox-identity
```

5. 授予使用 Secret Manager 的權限：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
    --role roles/secretmanager.secretAccessor
```

6. 將 AlloyDB 來源專屬的其他權限授予服務帳戶 (roles/alloydb.client 和 roles/serviceusage.serviceUsageConsumer)

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
    --role roles/alloydb.client

gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member serviceAccount:toolbox-identity@$PROJECT_ID.iam.gserviceaccount.com \
    --role roles/serviceusage.serviceUsageConsumer
```

7. 將 tools.yaml 上傳為密鑰：

```
gcloud secrets create tools-cymbal-transit --data-file=tools.yaml
```

8. 如果您已有密鑰，並想更新密鑰版本，請執行下列指令：

```
gcloud secrets versions add tools-cymbal-transit --data-file=tools.yaml
```

9. 將環境變數設為要用於 Cloud Run 的容器映像檔：

```
export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:latest
```

10. 執行下列指令，將 Toolbox 部署至 Cloud Run：

如果您已在 AlloyDB 執行個體中啟用公開存取權，請按照下列指令將應用程式部署至 Cloud Run：

```
gcloud run deploy toolbox-cymbal-transit \
  --image $IMAGE \
  --service-account toolbox-identity \
  --region us-central1 \
  --set-secrets "/app/tools.yaml=tools-cymbal-transit:latest" \
  --args="--tools-file=/app/tools.yaml","--address=0.0.0.0","--port=8080" \
  --allow-unauthenticated
```

如果您使用 **虛擬私有雲網路**，請使用下列指令：

```
gcloud run deploy toolbox-cymbal-transit \
  --image $IMAGE \
  --service-account toolbox-identity \
  --region us-central1 \
  --set-secrets "/app/tools.yaml=tools-cymbal-transit:latest" \
  --args="--tools-file=/app/tools.yaml","--address=0.0.0.0","--port=8080" \
  --network <<YOUR_NETWORK_NAME>> \
  --subnet <<YOUR_SUBNET_NAME>> \
  --allow-unauthenticated
```

注意：部署完成後，請前往 [Cloud Run 服務清單](#)，並確認該服務的安全防護分頁中已選取「允許公開存取」。

步驟 6 · 約 0 分鐘

6. 設定代理程式應用程式

將這個存放區複製到專案中，然後逐步瞭解。

[GitHub 存放區](#)

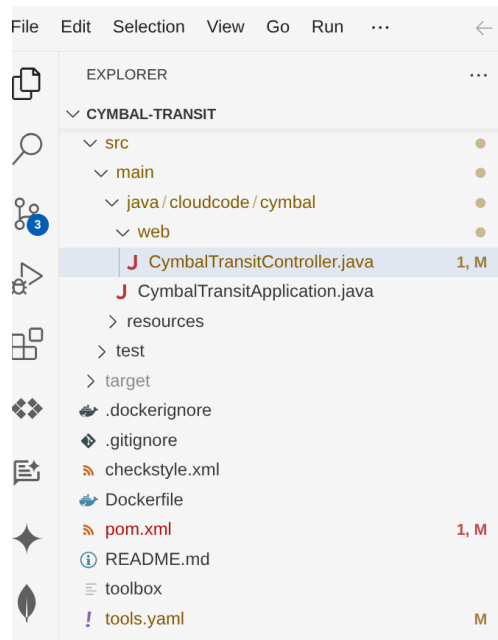
如要複製這個存放區，請在 Cloud Shell 終端機 (在根目錄或要建立這個專案的任何位置) 執行下列指令：

```
git clone https://github.com/googleapis/mcp-toolbox-sdk-java
```

上述指令實際上會複製整個 mcp-toolbox-sdk-java。我們只需要其中的範例專案。因此，請前往存放區內的專案根目錄：

```
cd mcp-toolbox-sdk-java/demo-applications/cymbal-transit
```

1. 這應該會建立專案，您可以在 Cloud Shell 編輯器中驗證。



2. 開啟 CymbalTransitController.java 並設定環境變數：

3. GCP_PROJECT_ID

4. GCP_REGION

5. GEMINI_MODEL_NAME

6. MCP_TOOLBOX_URL

或者 (僅限開發用途)，您也可以替換相應的回退值預留位置。

7. 程式碼逐步操作說明

`CymbalTransitController` 會做為 Cloud Run 服務的進入點。負責管理對話流程，並確保代理程式可存取使用者目前的要求。

實作方式採用分層架構，可分離 AI 自動化調度管理、工具橋接和低階 MCP 通訊。

1. AI 代理設定 (`AgentConfiguration`)

這個類別會使用 Spring 的 `@Configuration` 啟動 AI 元件。這會初始化 `VertexAiGeminiChatModel`，並將其繫結至我們的 Agent 介面。

```
@Bean
ChatLanguageModel geminiChatModel() {
    return VertexAiGeminiChatModel.builder()
        .project(projectId)
        .location(region)
        .modelName(modelName)
        .build();
}

@Bean
TransitAgent transitAgent(ChatLanguageModel chatLanguageModel, TransitAgentTools tools) {
    return AiServices.builder(TransitAgent.class)
        .chatLanguageModel(chatLanguageModel)
        .chatMemoryProvider(memoryId -> MessageWindowChatMemory.withMaxMessages(20))
        .tools(tools)
        .build();
}
```

重要性： `AiServices` 將介面繫結至 LLM。`MessageWindowChatMemory` 可確保代理程式在單一工作階段中，記住最多 20 則訊息的使用者偏好設定 (例如先前提到寵物提籠)。

2. AI 代理介面 (`TransitAgent`)

`@SystemMessage` 註解會定義「Persona」和作業限制，特別是路徑策略。

```
@SystemMessage({
    "You are the Cymbal Transit Concierge.",
    "CRITICAL INSTRUCTION: On your very first interaction, you MUST use the 'findAllSchedules' tool to fetch and memorize the broad bus routes.",
    "ONLY if the user asks a specifically narrowed-down question... should you route to the specific tools like 'querySchedules', 'bookTicket', 'searchPolicies'.",
    "Don't show any asterisks while listing results. Keep it formatted and numbered or bulleted."
})
String chat(@MemoryId String sessionId, @UserMessage String userMessage);
```

重要性： 這項策略可將延遲時間縮到最短。代理程式會先擷取廣泛資料，然後使用內部脈絡回答一般路徑問題，不必發出多餘的後端呼叫。

3. 工具箱橋接器 (`TransitAgentTools`)

這項服務是代理程式的「雙手」，可將 `LangChain4j` 工具呼叫轉換為執行邏輯。

```
@Tool("Fetches the initial, broad dataset of all available bus schedules and routes.")
public String findAllSchedules() {
    return mcpService.findAllSchedules().join();
}

@Tool("Book a ticket for a passenger using a specific trip ID.")
public String bookTicket(String tripId, String passengerName) {
    return mcpService.bookTicket(tripId, passengerName).join();
}
```

同步執行：雖然 MCP 呼叫是非同步 (傳回 `CompletableFuture`)，但 LLM 需要結果才能繼續「思考」程序。我們使用 `.join()` 將同步結果傳回給代理程式。

4. MCP 工具箱服務 (`McpToolboxService`)

這是通訊層，使用 MCP Toolbox Java SDK 與 AlloyDB 後端互動。

```
// Identity Management: Fetching OIDC ID Token for Auth
GoogleCredentials credentials = GoogleCredentials.getApplicationDefault();
this.idToken = ((IdTokenProvider) credentials)
    .idTokenWithAudience(targetUrl, Collections.emptyList())
    .getTokenValue();

// Dynamic Invocation: Executing a tool by name
public CompletableFuture<String> findAllSchedules() {
    return mcpClient.invokeTool("find-bus-schedules",
        Collections.emptyMap()).thenApply(result -> {
        return result.content().stream()
            .map(content -> content.text())
            .collect(Collectors.joining(", ", "[", "]"));
    });
}
```

重要性：`McpToolboxClient` 可處理 JSON-RPC 通訊的繁重工作。`bookTicket` 方法特別展示了 SDK 動態繫結複雜參數的能力。

5. REST 控制器 (`TransitAgentController`)

由於 `LangChain4j` 會管理狀態和邏輯，因此最終端點大幅簡化。

```
@PostMapping("/chat")
public ResponseEntity<String> handleUserChat(@RequestBody String userMessage, HttpSession
session) {
    String sessionId = session.getId();
    String agentResponse = transitAgent.chat(sessionId, userMessage);
    return ResponseEntity.ok(agentResponse);
}
```

重要性：將 `HttpSession` ID 對應至 `@MemoryId`，可確保不同使用者的旅遊行程不會混淆，同時保持控制器程式碼乾淨且易於閱讀。

8. MCP Toolbox：重要性和 Java SDK

什麼是 MCP？

Model Context Protocol (MCP) 就像是 AI 的通用翻譯器，MCP 的用途是將 AI 模型連結外部工具和資料集的程序標準化，以安全通用的通訊協定取代自訂的片段整合指令碼。無論代理程式需要執行交易式 SQL 查詢、搜尋數千份政策文件，還是觸發 REST API，MCP 都提供單一整合介面。

MCP Toolbox for Databases

工程團隊已不再使用簡單的聊天機器人，而是建構可直接與任務關鍵資料庫互動的代理系統。不過，建構這些企業代理程式通常意味著會遇到整合瓶頸，例如自訂黏著程式碼、脆弱的 API 和複雜的資料庫邏輯。

為以安全且統一的控制層取代這些硬式編碼瓶頸，我們很高興宣布推出資料庫適用的 Model Context Protocol (MCP) 工具箱 [Java SDK](#)。這項版本在全世界最廣為採用的企業生態系統中，提供一流的型別安全代理程式協調功能。Java 成熟的架構專為這些嚴格需求而設計，可提供高並行性、嚴格的交易完整性，以及強大的狀態管理功能，確保在正式環境中安全地擴充重要業務 AI 代理。

為何選擇 Java SDK？

MCP Toolbox Java SDK 可讓 Java 開發人員執行下列操作：

1. **使用工具**：連線至 MCP 伺服器 (例如 AlloyDB 適用的 MCP Toolbox)，並自動將伺服器的功能轉換為 LangChain4j 可理解的 Java 方法。
2. **型別安全**：針對工具參數運用 Java 的嚴格型別，減少工具呼叫中的執行階段「幻覺」錯誤。
3. **企業就緒**：輕鬆整合 Spring Boot、Quarkus、Micronaut 等。
4. **輕鬆連線**：不必編寫樣板 JSON-RPC 程式碼。
5. **標準化驗證**：原生支援 Google Cloud OIDC 權杖，確保工具安全執行。

[更多](#)。

依附元件：pom.xml 設定

將下列依附元件新增至 Maven 專案，即可納入最新版 MCP Toolbox Java SDK：

```
<dependency>
  <groupId>com.google.cloud.mcp</groupId>
  <artifactId>mcp-toolbox-sdk-java</artifactId>
  <version>0.2.0</version>
</dependency>
```

將下列依附元件新增至 Maven 專案，即可納入 LangChain4j 構件：

```
<!-- LangChain4j Core & Gemini -->
<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j</artifactId>
  <version>0.35.0</version>
</dependency>
```

這樣就大功告成了！我們已成功複製專案，並詳細說明代理程式、MCP Toolbox Java SDK 和內容。

步驟 9 · 約 0 分鐘

9. 在本機執行

如要在機器上測試代理，您需要將代理指向已部署的 MCP Toolbox 伺服器。

1. 設定環境變數：

```
export GCP_PROJECT_ID="<<YOUR_PROJECT_ID>>"
export GCP_REGION="us-central1"
export GEMINI_MODEL_NAME="gemini-2.5-flash"
export MCP_TOOLBOX_URL="<<YOUR_TOOLBOX_ENDPOINT_URL>>/mcp"
```

2. 使用 Maven 執行：

```
mvn compile

mvn spring-boot:run
```

這項指令會在本地啟動代理程式，您應該可以測試。

步驟 10 · 約 0 分鐘

10. 將其部署至 Cloud Run

在專案複製完成的 Cloud Shell 終端機中執行下列指令，將其部署至 Cloud Run。請務必位於專案的根資料夾中。

如果不在目前專案的根資料夾中，請在 Cloud Shell 終端機中執行下列指令：

```
cd cymbal-transit
```

如果您已位於 cymbal-transit 根目錄，請執行下列指令，直接將應用程式部署至 Cloud Run：

```
gcloud run deploy cymbal-transit --source . --set-env-vars GCP_PROJECT_ID=
<<YOUR_PROJECT_ID>>,GCP_REGION=us-central1,GEMINI_MODEL_NAME=gemini-2.5-
flash,MCP_TOOLBOX_URL=<<YOUR_MCP_TOOLBOX_URL>> --allow-unauthenticated
```

將預留位置 <<YOUR_PROJECT>> and <<YOUR_MCP_TOOLBOX_URL>> 的值替換為實際值

指令完成後，會輸出服務網址。複製。

將 **AlloyDB Client** 角色授予 Cloud Run 服務帳戶。這樣一來，無伺服器應用程式就能安全地連線至資料庫。

在 Cloud Shell 終端機中執行下列指令：

```
# 1. Get your Project ID and Project Number
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")

# 2. Grant the AlloyDB Client role
gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
--role="roles/alloydb.client"
```

注意：部署完成後，請前往 [Cloud Run 服務清單](#)，並確認該服務的安全防護分頁中已選取「允許公開存取」。

現在請使用服務網址 (先前複製的 Cloud Run 端點) 測試應用程式。

注意：如果遇到服務問題，且系統指出原因為記憶體不足，請嘗試將分配的記憶體上限提高至 1 GiB，然後進行測試。

11. 示範

詢問服務專員：「我明天早上需要從紐約前往波士頓。我可以帶黃金獵犬來嗎？」觀察代理如何：

- 1. 搜尋大型犬的政策。
- 2. 尋找特定行程。
- 3. 以行程 ID 歸納最快的行程。
- 4. 如果你後續採取該動作要求，系統也會預訂票證。

 **Cymbal Transit Concierge**
Powered by AlloyDB & MCP Toolbox



Hello! I'm the Cymbal Transit Concierge. I can help you find bus schedules, book tickets, or answer questions about our travel policies. Where are you heading today?



hi I need to get from New York to Boston tomorrow morning. Can I bring my Golden Retriever?



Here are the available trips from New York to Boston for tomorrow morning (February 25, 2026):

1. **Trip ID:** cab62dc3-e5cb-4cf4-bb75-158e9762cd65

Departure: 2026-02-25 at 04:00 AM (EST)

Arrival: 2026-02-25 at 08:30 AM (EST)

Ticket Price: \$52.00

Available Seats: 48

2. **Trip ID:** 930f5038-2f7c-43c8-9abc-6b253d0148cc

Departure: 2026-02-25 at 08:00 AM (EST)

Arrival: 2026-02-25 at 12:30 PM (EST)

Ticket Price: \$25.96

Available Seats: 34

Regarding bringing your Golden Retriever, our policy states that large dogs are not permitted on standard coaches. Service animals are always welcome. Small pets (under 25 lbs) are allowed in secure carriers for a \$25 fee.

步驟 12 · 約 0 分鐘

12. 清除所用資源

完成本實驗室後，別忘了刪除 AlloyDB 叢集和執行個體。

前往 <https://console.cloud.google.com/alloydb/clusters>。按一下要刪除的叢集旁的垂直省略號，然後按一下「刪除」。

這項作業應會清理叢集及其執行個體。

步驟 13 · 約 0 分鐘

13. 恭喜

您已成功建構以 Java 為基礎的複雜轉運代理程式。您運用 LangChain4j 進行自動化調度管理，並使用 MCP Toolbox Java SDK 連結資料，建立的系統可跨代理、工具和資料來源進行推理。如要開始使用 MCP Toolbox for Databases 協調多個資料庫 (甚至是跨平台) 的代理式應用程式，請立即使用 [Java SDK](#)！請參閱這篇發布公告網誌文章，進一步瞭解程式庫。如要親自建構更多這類應用程式，並在講師的帶領下免費學習，請前往 <https://codevipassana.dev> 報名參加 Code Vipassana！
